Videogame Review API

API documentation for endpoints for a RESTful Videogame Review API.

Deployed at https://hoeyinfoong.pythonanywhere.com/api

POST /token/

https://hoeyinfoong.pythonanywhere.com/api/token/

JSON Request Body

Structure

- `username` : string - unique identifier for the user being authenticated.
- `password` : string - corresponding password for the username provided.

Example

```json
{
    "username": "admin",
    "password": "examplepassword"
}
```

JSON Response

Structure

- `access` : string - a JSON Web Token that can be used to authenticate the user's identity. without providing a **username** and **password**. Valid for 5 minutes.
- `refresh` : string - a JSON Web Token that can be used to provide a new access token via `POST api/token/refresh/` without needing to provide a **username** and **password**. Valid for 1 day.

Example

```json
{
    "access": "exampleaccesstoken12345",
    "refresh": "examplerefreshtoken67890"
}
```

Description

Returns two JSON Web Tokens as described above, using the provided **username** and **password** in the body. If successful, returns a `HTTP 200 Ok`. Body must have Content-Type `application/json`.

If **username** and/or **password** is incorrect, returns a `HTTP 401 Unauthorized`.

If **username** or **password** is missing, returns a `HTTP 400 Bad Request.`

The access token is needed to perform the following API endpoints:

- `POST /reviews/`
- `PATCH /reviews/{review_id}`
- `DELETE /reviews/{review_id}`

HEADERS

Content-Type	application/json

POST /token/refresh/

https://hoeyinfoong.pythonanywhere.com/api/token/refresh/

JSON Request Body

Structure

- `refresh` : string - a valid refresh JSON Web Token for the user that wants reauthenticate.

Example

```
json

{"refresh": "examplerefreshtoken67890"}
```

JSON Response

Structure

- `access` : string - a JSON Web Token that can be used to authenticate the user's identity. without providing a **username** and **password**. Valid for 5 minutes.

Example

```
json
```

```
{"access": "exampleofnewaccesstoken"}
```

Description

Returns an **access** JSON Web Token as described above, using the provided **refresh** JSON Web Token. If successful, returns a `HTTP 200 Ok`. Body must have Content-Type `application/json`.

If **refresh** is invalid, returns a `HTTP 401 Unauthorized`.

If **refresh** is missing, returns a `HTTP 400 Bad Request`.

The access token is needed to perform the following API endpoints:

- `POST /reviews/`
- `PATCH /reviews/{review_id}`
- `DELETE /reviews/{review_id}`

HEADERS

Content-Type	application/json

POST /users/

https://hoeyinfoong.pythonanywhere.com/api/users/

JSON Request Body

Structure

- `username` : string - must be unique among all users.

- `password` : string - must be at least 12 characters long, non-numeric, and not too similar to the **username**.

Example

```json
{
    "username": "new_user",
    "password": "examplepassword"
}
```

JSON Response

Structure

- `username` : string - newly created user's username (matching **username** in request body).
- `url` : link - retrieves new user's profile via `GET /users/{user_id}`

Example

```json
{
    "username": "new_user",
    "url": "https://hoeyinfoong.pythonanywhere.com/api/users/12345678"
}
```

Description

Creates a new user using the provided **username** and **password** in the request body. If successful, returns a `HTTP 201 Created`.

If **username** is already taken, returns a `HTTP 409 Conflict`.

If **password** is not valid, returns a `HTTP 400 Bad Request`.

If provided body is not valid (missing or misnamed keys), returns a `HTTP 400 Bad Request`.

Passwords are encrypted using a PBKDF2 algorithm with a SHA256 hash before storage.

HEADERS

Content-Type	application/json

GET /users/{user_id}

https://hoeyinfoong.pythonanywhere.com/api/users/:user_id

JSON Response

Structure

- `id` : integer - user id matching **user_id** path variable.
- `username` : string - user's username.
- `url` : link - redundant link to this resource.
- `reviews` : link - retrieves this user's reviews via `GET /users/{user_id}/reviews/`
- `reviews_n` : integer - number of reviews the user has authored.
- `average_score` : integer - user's average review score out of 100, rounded down to nearest integer.
- `reviews_hist` : list - review score distribution, grouping scores in intervals of 20. For example `[2, 5, 10, 60, 20]` means there are two reviews with a score 0-20, five reviews with a score 21-40, ten reviews with a score 41-60, sixty reviews with a score 61-80, and twenty reviews with a score 81-100.
- `favroite_genres` : list - user's three favorite genres, ordered by their average score for games in each genre (for genres with at least 1 review).

- `favorite_games` : list - user's three best reviews, ordered by highest review score.
  - `url` : link - retrieves the review via `GET /reviews/{review_id}`
  - `game` : representation of the game
    - `title` : string - game title.
    - `url` : link - retrieves the game via `GET /games/{game_id}`
  - `score` : integer - score out of 100.

Example

`GET /users/3137`


```
{
"id": 3137,
"username": "Broyax",
"url": "https://hoeyinfoong.pythonanywhere.com/api/users/3137",
"reviews": "https://hoeyinfoong.pythonanywhere.com/api/users/3137/reviews/",
"reviews_n": 53,
"average_score": 36,
```

```
"reviews_hist": [
21,
11,
9,
9,
3
],
```

```
"favorite_genres": [
"Western RPG",
"Action RPG",
"FPS"
],
```

```
"favorite_games": [
{
"url": "https://hoeyinfoong.pythonanywhere.com/api/reviews/42644",
"game": {
"title": "Mass Effect 2",
```

"url": "https://hoeyinfoong.pythonanywhere.com/api/games/24"
},
"score": 100
},
{
"url": "https://hoeyinfoong.pythonanywhere.com/api/reviews/108394",
"game": {
"title": "Star Wars: Knights of the Old Republic",
"url": "https://hoeyinfoong.pythonanywhere.com/api/games/85"
},
"score": 100
},
{
"url": "https://hoeyinfoong.pythonanywhere.com/api/reviews/129058",
"game": {
"title": "Mass Effect 3",
"url": "https://hoeyinfoong.pythonanywhere.com/api/games/117"
},
"score": 90
}
]
}

Description

Returns a user profile (matching **user_id** path variable) with statistics based on the user's reviews, and a `HTTP 200 OK`.

If **user_id** is invalid or missing, returns a `HTTP 404 Not Found`.

PATH VARIABLES

user_id	integer - user id

GET /users/{user_id}/reviews/

https://hoeyinfoong.pythonanywhere.com/api/users/:user_id/reviews/

JSON Response

Structure

List of reviews, ordered by most recently published (limit of 25 each page):

- `id` : integer - review id.
- `url` : link - retrieves the specific review via `GET /reviews/{review_id}`
- `user` : representation of the author of the review.
  - `username` : string - author's username.
  - `url` : link - retrieves the author's profile via `GET /users/{user_id}`
- `game` : representation of the game that the review is reviewing.
  - `title` : string - game title.
  - `url` : link - retrieves the game via `GET /games/{game_id}`
- `date` : date YYYY-MM-DD - date the review was published.
- `score` : integer - review score out of 100.
- `content` : string - content of review.

Example

`GET /users/3137/reviews/` : Note that this only shows the first two reviews for the user as an example.


```
[
{
"id": 95325,
"url": "https://hoeyinfoong.pythonanywhere.com/api/reviews/95325",
"user": {
"username": "Broyax",
```

"url": "https://hoeyinfoong.pythonanywhere.com/api/users/3137"
},
"game": {
"title": "Metaphor: ReFantazio",
"url": "https://hoeyinfoong.pythonanywhere.com/api/games/67"
},
"date": "2024-12-14",
"score": 0,
"content": "Sous ce titre un peu étrange et ronflant se cache en vérité un Persona rebadgé et (mal) recarrossé, mal retapé et mal dégrossi... un fiasco !"
},

{
"id": 87578,
"url": "https://hoeyinfoong.pythonanywhere.com/api/reviews/87578",
"user": {
"username": "Broyax",
"url": "https://hoeyinfoong.pythonanywhere.com/api/users/3137"
},
"game": {
"title": "God of War",
"url": "https://hoeyinfoong.pythonanywhere.com/api/games/52"
},
"date": "2024-11-17",
"score": 0,
"content": "Déjà le titre (comme beaucoup d'autres avant lui et bien après encore -hélas) est le signe patent d'une paresse intellectuelle... jeu de merde !"
},

Description

Returns a list of the user's reviews specified by the **user_id** path variable, and a `HTTP 200 OK`.

An invalid or empty user**_id** will return a `HTTP 404 Not Found`.

There is a limit of 25 reviews per page, and additional reviews can be found using the **page** query parameter: `GET /games/{game_id}/reviews/?page=2`. An invalid **page** will return a `HTTP 404 Not Found`.

PARAMS

page	page number for list of reviews (integer)

PATH VARIABLES

user_id	integer - user id

GET /games/

https://hoeyinfoong.pythonanywhere.com/api/games/

JSON Response

Structure

- `analytics` : simple analysis on the list of games retrieved by the request.
  - `top_rated` : list - the top 5 games, sorted by highest average score.
    - `game` : representation of a game.
      - `title` : string - game title.
      - `url` : link - retrieves specific game via `GET /games/{game_id}`
    - `average_score` : integer - average score of the game out of 100, rounded down to the nearest integer.
  - `trending` : list - the top 5 games, sorted by the number of reviews from the most recent 100 reviews for this list of games.
    - `title` : string - title of the game.
    - `url` : link - retrieves the specific game via `GET /games/{game_id}`

- `games` : list - games retrieved by the request, sorted by most recent release date. Limit of 25 per page.
  - `id` : integer - game id.
  - `title` : string - game title.
  - `url` : link - retrieves the specific game via `GET /games/{game_id}`
  - `release_date` : date YYYY-MM-DD - release date of the game.

Example without query parameters

`GET /games/` : Only shows the first two games in the games list as an example


```
{
"analytics": {
"top_rated": [
{
"game": {
"title": "Disco Elysium: The Final Cut",
"url": "https://hoeyinfoong.pythonanywhere.com/api/games/495"
},
"average_score": 100
},
{
"game": {
"title": "The Last of Us Part II",
"url": "https://hoeyinfoong.pythonanywhere.com/api/games/91"
},
"average_score": 99
},
{
"game": {
"title": "NFL 2K1",
"url": "https://hoeyinfoong.pythonanywhere.com/api/games/15"
},
"average_score": 98
},
{
"game": {
```

"title": "Astro Bot",
"url": "https://hoeyinfoong.pythonanywhere.com/api/games/55"
},
"average_score": 96
},
{
"game": {
"title": "The Legend of Zelda Collector's Edition",
"url": "https://hoeyinfoong.pythonanywhere.com/api/games/39"
},
"average_score": 95
}
],

"trending": [
{
"title": "Astro Bot",
"url": "https://hoeyinfoong.pythonanywhere.com/api/games/55"
},
{
"title": "The Legend of Zelda: Tears of the Kingdom",
"url": "https://hoeyinfoong.pythonanywhere.com/api/games/29"
},
{
"title": "Elden Ring",
"url": "https://hoeyinfoong.pythonanywhere.com/api/games/26"
},
{
"title": "Red Dead Redemption 2",
"url": "https://hoeyinfoong.pythonanywhere.com/api/games/9"
},
{
"title": "The Legend of Zelda: Ocarina of Time",
"url": "https://hoeyinfoong.pythonanywhere.com/api/games/1"
}
]
},

```
"games": [
{
"id": 2465,
"title": "WWE 2K25",
```

```
},
{
"id": 1665,
"title": "Expelled!",
"url": "https://hoeyinfoong.pythonanywhere.com/api/games/1665",
"release_date": "2025-03-12"
},
...
]
```


Description

Returns an analysis of and list of all games in the database, and a `HTTP 200 OK`. Can return a `HTTP 404 Not Found` if query parameter **page** (see the rest of this section) is invalid.

The list of games is paginated to display 25 games per page. For example, to view games 26-50 in the second page, use query parameters: `GET /games/?page=2`.

Other query parameters (defined in the section below) can be specified to filter to list of games retrieved. The analysis will be based on games filtered by all query parameters excluding **page**, e.g. `GET /games/?genre=action&platform=pc&page=2` contains the analysis of all action games on the PC platform. Thus between pages, the analysis for each page will remain the same, while only the list of games changes.

Invalid query parameters will return a `HTTP 400 Bad Request`

PARAMS

page	integer - page number for list of games

title	string - filter games by title (case-insensitive search for titles that contain this value)
genre	string - filter games by genre (case-insensitive exact match)
platform	string - filter games by platform (case-insensitive exact match)
developer	string - filter games by developer (case-insensitive exact match)
publisher	string - filter games by publisher (case-insensitive exact match)

GET /games/{game_id}

https://hoeyinfoong.pythonanywhere.com/api/games/:game_id

JSON Response

Structure

- `id` : integer - game id matching **game_id** path variable.
- `title` : string - game title.
- `url` : link - for this resource.
- `release_date` : date YYYY-MM-DD - release date of this game.
- `rating` : string - maturity rating of this game.
- `description` : string - game description
- `genre` : string - game genre.
- `platforms` : list - platforms this game can be played on.
- `developers` : list - developers that made this game.
- `publishers` : list - publishers that published this game.
- `reviews` : link - retrieves this game's reviews via `GET /games/{game_id}/reviews`
- `analytics` : link - retrives this game's analytics via `GET /games/{game_id}/analytics`

Example

`GET /games/50`


```
{
"id": 50,
"title": "Portal 2",
"url": "https://hoeyinfoong.pythonanywhere.com/api/games/50",
"release_date": "2011-04-19",
"rating": "E10+",
"description": "Valve is working on a full-length sequel to its award-winning, mind-bending puzzle action game for the Xbox 360 and PC.",
"genre": "FPS",
"platforms": [
"Xbox 360",
"PlayStation 3",
"PC"
],
"developers": [
"Valve Software"
],
"publishers": [
"Electronic Arts",
"Valve Software"
],
"reviews": "https://hoeyinfoong.pythonanywhere.com/api/games/50/reviews/",
"analytics": "https://hoeyinfoong.pythonanywhere.com/api/games/50/analytics"
}
```


Description

Returns the details of the game resource specified by the **game_id** path variable, and a `HTTP 200 OK`.

An invalid **game_id** will return a `HTTP 404 Not Found`.

An empty **game_id** will use the `GET /games/` endpoint instead.

PATH VARIABLES

game_id	integer - game id

GET /games/{game_id}/reviews/

https://hoeyinfoong.pythonanywhere.com/api/games/:game_id/reviews/

JSON Response

Structure

List of reviews, ordered by most recently published (limit of 25 each page):

- `id` : integer - review id.
- `url` : link - retrieves the specific review via `GET /reviews/{review_id}`
- `user` : representation of the author of the review.
  - `username` : string - author's username.
  - `url` : link - retrieves the author's profile via `GET /users/{user_id}`
- `game` : representation of the game that the review is reviewing.
  - `title` : string - game title.
  - `url` : link - retrieves the game via `GET /games/{game_id}`
- `date` : date YYYY-MM-DD - date the review was published.
- `score` : integer - review score out of 100.
- `content` : string - content of review.

Example

`GET /games/50/reviews/` : Only the first two are shown as an example.

[
{
"id": 80871,
"url": "https://hoeyinfoong.pythonanywhere.com/api/reviews/80871",
"user": {
"username": "convretcetaste",
"url": "https://hoeyinfoong.pythonanywhere.com/api/users/21964"
},
"game": {
"title": "Portal 2",
"url": "https://hoeyinfoong.pythonanywhere.com/api/games/50"
},
"date": "2025-03-08",
"score": 80,
"content": "creative and innovative for it's time, has a fun gameplay and co op mode, a good story, solid game overall\n\n8.5"
},

{
"id": 80759,
"url": "https://hoeyinfoong.pythonanywhere.com/api/reviews/80759",
"user": {
"username": "Sweetfr19",
"url": "https://hoeyinfoong.pythonanywhere.com/api/users/18199"
},
"game": {
"title": "Portal 2",
"url": "https://hoeyinfoong.pythonanywhere.com/api/games/50"
},
"date": "2025-03-04",
"score": 100,
"content": "Best Valve Game I ever Played. Good Story. Good Puzzle. Good Map Design. Wishing For Portal 3."
},
...
]

Description

Returns a list of the game's reviews specified by the **game_id** path variable, and a `HTTP 200 OK`.

An invalid or empty **game_id** will return a `HTTP 404 Not Found`.

There is a limit of 25 reviews per page, and additional reviews can be found using the **page** query parameter: `GET /games/{game_id}/reviews/?page=2`. An invalid **page** will return a `HTTP 404 Not Found`.

PARAMS

page	page number for list of reviews (integer)

PATH VARIABLES

game_id	integer - game id

GET /games/{game_id}/analytics

https://hoeyinfoong.pythonanywhere.com/api/games/:game_id/analytics

JSON Response

Structure

- `game` : representation of the game that has been analysed.
  - `title` : string - game title.
  - `url` : link - retrieves the game via `GET /games/{game_id}`
- `reviews_n` : integer - number of reviews for the game.

- `average_score` : integer - average review score out of 100, rounded down to nearest integer.
- `reviews_hist` : list - review score distribution, grouping scores in intervals of 20. For example `[2, 5, 10, 60, 20]` means there are two reviews with a score 0-20, five reviews with a score 21-40, ten reviews with a score 41-60, sixty reviews with a score 61-80, and twenty reviews with a score 81-100.
- `best_month` : representation of the month the game was most popular, determined by the month it received the most reviews.
  - `date` : date YYYY-MM-DD - representation of the month the game was most popular.
  - `reviews_n` : integer - number of reviews published that month for the game.

Example

`GET /games/50/analytics`


```
{
"game": {
"title": "Portal 2",
"url": "https://hoeyinfoong.pythonanywhere.com/api/games/50"
},
"reviews_n": 3046,
"average_score": 93,
"reviews_hist": [
39,
19,
70,
290,
2628
],
"best_month": {
"date": "2011-04-19",
"reviews_n": 451
}
}
```


Description

Returns a simple analysis of the game matching the **game_id** path variable, and a `HTTP 200 OK`.

An invalid or empty **game_id** will return a `HTTP 404 Not Found`.

PATH VARIABLES

game_id	integer - game id

GET /reviews/{review_id}

https://hoeyinfoong.pythonanywhere.com/api/reviews/:review_id

JSON Response

Structure

- `id` : integer - review id matching **review_id** path variable.
- `url` : link - for this resource.
- `user` : representation of author of the review
  - `username` : string - author's username.
  - `url` : link - retrieves the author via `GET /users/{user_id}`
- `game` : representation of game the review is reviewing.
  - `title` : string - game title.
  - `url` : link - retrieves the game via `GET /games/{game_id}`
- `date` : date YYYY-MM-DD - date that review was published.
- `score` : integer - review score out of 100.
- `content` : string - content of review.

Example

`GET /reviews/1`


```
{
"id": 1,
"url": "https://hoeyinfoong.pythonanywhere.com/api/reviews/1",
"user": {
"username": "Filipe69",
"url": "https://hoeyinfoong.pythonanywhere.com/api/users/2"
},
"game": {
"title": "The Legend of Zelda: Ocarina of Time",
"url": "https://hoeyinfoong.pythonanywhere.com/api/games/1"
},
"date": "2023-01-30",
"score": 100,
"content": "C'est jeu est le meilleur jeu qui ai pu existé, une histoire incroyable des boss et un game play parfait !"
}
```


Description

Returns the details of the review specified by the **review_id** path variable, and a `HTTP 200 OK`.

If **review_id** is invalid or missing, returns a `HTTP 404 Not Found`.

PATH VARIABLES

review_id	integer - review id

POST /reviews/

https://hoeyinfoong.pythonanywhere.com/api/reviews/

JSON Request Body

Structure

- `game` : integer - game id
- `date` : date YYYY-MM-DD - date to publish review. Must be today or earlier.
- `score` : integer - score out of 100.
- `content` : string - content of the review. Optional.

Example

```json
{
    "game": 22,
    "date": "2022-22-22",
    "score": 22,
    "content": "This is an example of a request body for review creation."
}
```

JSON Response

Structure

The same as `GET /reviews/{review_id}` for the created review.

- `id` : integer - review id
- `url` : link - retrieves the new review via `GET /reviews/{review_id}`
- `user` : representation of author of the review
  - `username` : string - author's username.
  - `url` : link - retrieves the author via `GET /users/{user_id}`
- `game` : representation of game the review is reviewing, matching **game** in the request body,
  - `title` : string - game title.

    - `url` : link - retrieves the game via `GET /games/{game_id}`
- `date` : date YYYY-MM-DD - date review was published, matching **date** in request body.
- `score` : integer - review score out of 100, matching **score** in request body.
- `content` : string - content of review, matching **content** in request body if it was provided; else it is an empty string.

Example

```
{
"id": 12345,
"url": "https://hoeyinfoong.pythonanywhere.com/api/reviews/12345",
"user": {
"username": "user",
"url": "https://hoeyinfoong.pythonanywhere.com/api/users/67890"
},
"game": {
"title": "The Orange Box",
"url": "https://hoeyinfoong.pythonanywhere.com/api/games/22"
},
"date": "2022-22-22",
"score": 22,
"content": "This is an example of a request body for review creation."
}
```

Description

Creates a new review using the provided body data, and if successful returns a `HTTP 201 Created`. A valid **access** JSON Web Token must be provided in the request header (see next section). The token can be retrieved from `POST /token/` with a **username** and **password**, or from `POST /token/refresh/` if the user has a valid **refresh** JSON Web Token.

If the user is unauthenticated (missing or invalid **access** JSON Web Token), returns a `401 HTTP Unauthorized`.

If the authenticated user already has an existing review for the provided **game**, returns a `HTTP 409 Conflict`. Note that while the existing database has some users with multiple reviews for the same game, new reviews must adhere to this constraint (one review per user-game pair).

If **game** is invalid, returns a `HTTP 404 Not Found`.

If body data is incorrect or missing, returns a `HTTP 400 Bad Request`.

HEADERS

Authorization	Must be in the form "Bearer {jwt}", where {jwt} is a valid access JSON Web Token for the user creating the review.
Content-Type	application/json

PATCH /reviews/{review_id}

https://hoeyinfoong.pythonanywhere.com/api/reviews/:review_id

JSON Request Body

Structure

- `id` : integer - review id matching **review_id** path variable.
- `score` : integer - new score out of 100. Optional if **content** was provided.
- `content` : string - new content of the review. Optional if **score** was provided.

Example

```json
{
    "id": 12345,
    "score": 100,
```

```
    "content": "This is an example of an update to a review."
}
```

Description

Updates an existing review matching the **review_id** path variable, using the provided body data. If successful returns a `HTTP 204 No Content`. A valid **access** JSON Web Token must be provided in the request header (see next section). The token can be retrieved from `POST /token/` with a **username** and **password**, or from `POST /token/refresh/` if the user has a valid **refresh** JSON Web Token.

If the user is unauthenticated (missing or invalid **access** JSON Web Token), returns a `HTTP 401 Unauthorized`.

If user is authenticated but trying to `PATCH` a review they did not author, returns a `HTTP 403 Forbidden`.

If **review_id** is invalid, returns a `HTTP 404 Not Found`.

If **review_id** is missing, returns a `HTTP 400 Bad Request`.

If body data is incorrect or missing, returns a `HTTP 400 Bad Request`.

HEADERS

Authorization	Must be in the form "Bearer {jwt}", where {jwt} is a valid access JSON Web Token for the author of the review.
Content-Type	application/json

PATH VARIABLES

review_id	integer - review id

DELETE /reviews/{review_id}

https://hoeyinfoong.pythonanywhere.com/api/reviews/:review_id

Description

Deletes an existing review matching the **review_id** path variable. If successful returns a `HTTP 204 No Content`. A valid access JSON Web Token must be provided in the request header (see next section). The token can be retrieved from `POST /token/` with a **username** and **password**, or from `POST /token/refresh/` if the user has a valid **refresh** JSON Web Token.

If the user is unauthenticated (missing or invalid access JSON Web Token), returns a `HTTP 401 Unauthorized`.

If user is authenticated but trying to `DELETE` a review they did not author, returns a `HTTP 403 Forbidden`.

If **review_id** is invalid, returns a `HTTP 404 Not Found`.

If **review_id** is missing, returns a `HTTP 400 Bad Request`.

HEADERS

Authorization	Must be in the form "Bearer {jwt}", where {jwt} is a valid access JSON Web Token for the author of the review.
Content-Type	application/json

PATH VARIABLES

review_id integer - review id